Tema 6

Búsqueda con espacios estructurados

“Restricciones”

Año Académico 2026/27

Módulo II: Agentes basados en Búsqueda

Tema 2: Búsqueda no-informada

Tema 3: Búsqueda heurística

Tema 4: Búsqueda heurística avanzada

Tema 5: Búsqueda multiagente

Tema 6: Búsqueda con espacios estructurados

Módulo II: Agentes basados en Búsqueda

Tema 2: Búsqueda no-informada

Tema 3: Búsqueda heurística

Tema 4: Búsqueda heurística avanzada

Tema 5: Búsqueda multiagente

Tema 6: Búsqueda con espacios estructurados

- CSPs (problemas de satisfacción de restricciones)
- Satisfacción de restricciones
- Propagación de restricciones

Problemas de representaciones “monolíticas” de estados

- *representación* de los conocimientos a priori del agente
  - estado inicial: símbolo (s_0)
  - estados sucesores: función *expandir* ($s_7 \mapsto \{s_8, s_{10}, s_{12}, s_{27}, s_{112}\}$)
  - estados meta: función *meta?* ($s_{112} \mapsto$ true)
  - coste de operador: función ($c(s_7, s_{112}) \mapsto 5$)
  - heurísticas: función h^* ($s_7 \mapsto 235$)
- se abstrae completamente de las características de los estados
  - ¿bloque *A* encima de bloque *B*?, ¿el agente se encuentra en *Radovac*?, …
- Problema:
  - complejidad de la representación del conocimiento (p.e. *expandir*)
  - mantenibilidad del conocimiento
- Solución:
  - tomar en cuenta la “estructura” de los estados
  - definir un estado no por “un nombre” sino por el conjunto de sus *propiedades*

Problemas de Satisfacción de Restricciones

Estructura de estados:

- Un estado está compuesto por un conjunto de *n* variables que pueden tomar diferentes valores
- Un estado es un estado meta si los valores que tienen sus variables cumplen una serie de *restricciones*
- Muchos dominios reales se pueden describir de esta forma: juegos unipersonales (Sudoku), generación de horarios en institutos, planificación de la producción, etc.

Definición:

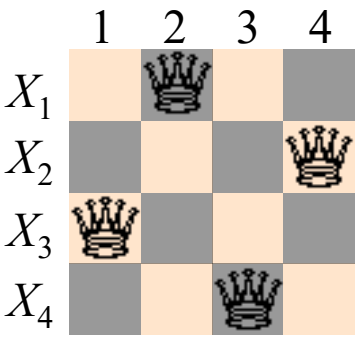
- Un **problema de satisfacción de restricciones** (*Constraint Satisfaction Problem,* ***CSP***) es una tripleta (X, D, R)
  - $X = \{x_1,\ldots, x_n\}$ es un conjunto finito de *variables*
  - D: $X \rightarrow V$ es una función total que asigna un *dominio* (conjunto finito de valores de V_i) a cada variable. Frecuentemente se escribe D_i en vez de $D(x_i)$ para referirse al dominio de la variable x_i
  - $R = \{R_1,\ldots, R_k\}$ es un conjunto finito de *restricciones* tal que cada R_i es un predicado sobre un subconjunto de las variables de X. Formalmente: $R_i(x_1,\ldots, x_l) \subseteq D_1\times\ldots\times D_l$

Ejemplo: *n*-reinas

Problema de las *n* reinas:

- Posicionar *n* reinas en un tablero $n \times n$, tal que ninguna de ellas está amenazada por otra

Ejemplo: **4 reinas**



4-reinas como CSP:

- $X = \{x_1,\ldots, x_4\}$ (número de las filas)
- $D = \{D_1,\ldots, D_4\}$ siendo $D_i = \{1,2,3,4\}$ (nº de las columnas)
- $R = \{R_1\}$ siendo $R_1(x_1, x_2, x_3, x_4) = \{(2,4,1,3),(3,1,4,2)\}$

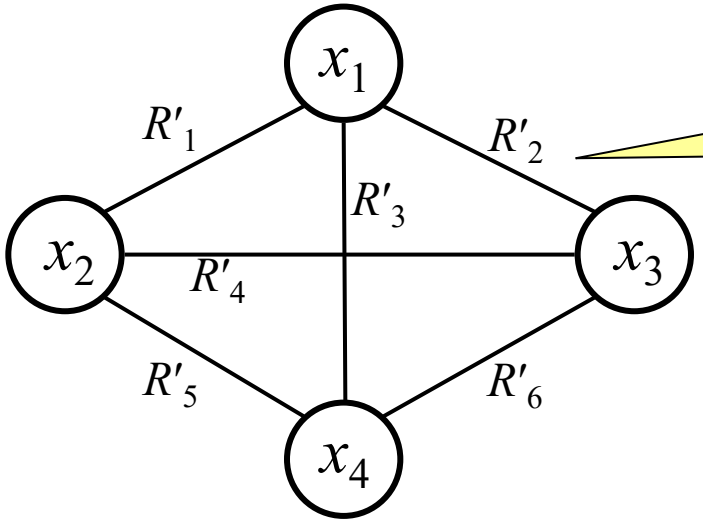
Nótese:

- *R* suele modelarse como *conjunto* de restricciones más "locales"
- En el ejemplo: $R' = \{R'_1,\ldots, R'_6\}$ refiriéndose cada R'_i a la amenaza entre dos filas
  - Ejemplo: R'_2 expresa que no debe haber amenaza entre reinas en las filas 1 y 3
  - $R'_2(x_1,x_3) = \{(1,2),(1,4),(2,1),(2,3),(3,2),(3,4),(4,1),(4,3)\}$
- En un CSP *binario* todas las restricciones involucran como mucho dos variables

CSP binario como grafo

Los CSPs de restricciones binarias se suelen representar como **grafos**

- Cada variable x_i del CSP es representada por un nodo
- Cada restricción binaria $R_i(x_j,x_k)$ se representa por un arco ente los nodos x_j y x_k
- Ejemplo: grafo correspondiente al problema de las 4-reinas



Restricción R'_2: " no debe haber amenaza entre reinas en las filas 1 y 3"

Ejemplo: *criptoaritmética*

Problema de criptoaritmética:

- Operación aritmética con letras
- Asignar a cada letra un dígito diferente de modo que se cumpla la operación

Ejemplo:

```
  TWO
+ TWO
  C3C2C1
-------
 FOUR
```

Ejemplo como CSP

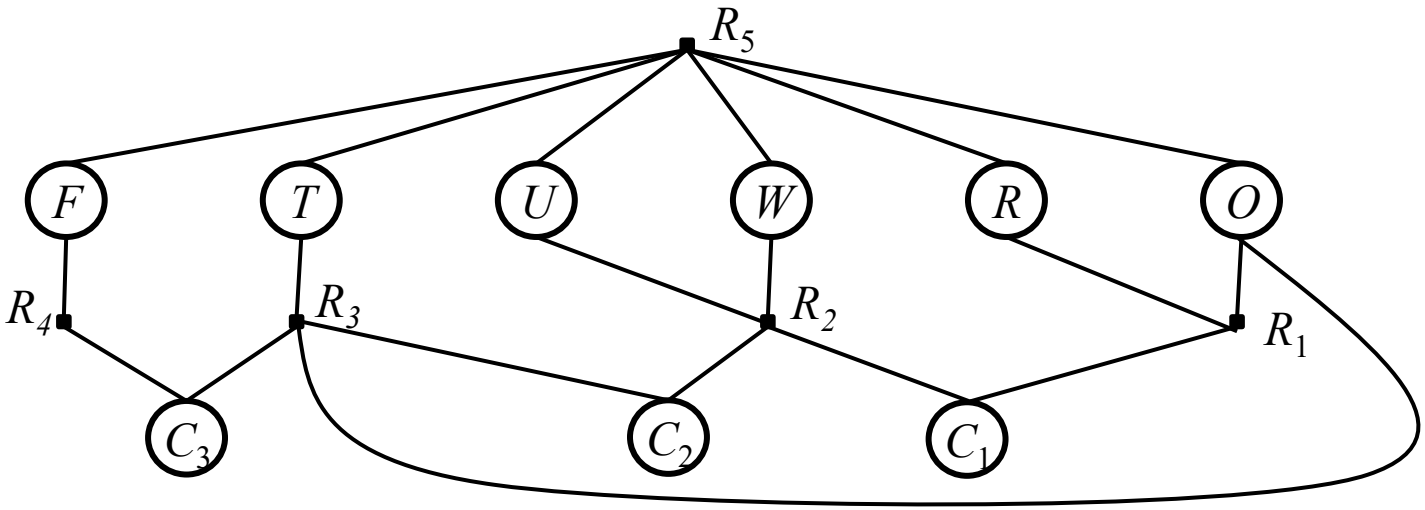
- Idea: variables auxiliares para el acarreo
- $X = \{T, W, O, R, U, F, C_1, C_2, C_3\}$
- $D_T = D_W = D_O = D_R = D_U = \{1,2,3,4,5,6,7,8,9\}$; $D_F = D_{C1} = D_{C2} = D_{C3} = \{0,1\}$
- $R = \{R_1(O,R,C_1), R_2(W,C_1,U,C_2), R_3(T,C_2,O,C_3), R_4(F,C_3), R_5(T,W,O,R,U,F)\}$
  - $R_1 = \{(O,R,C_1)$: $\boldsymbol{O+O = R + 10*C_1}\}$
  - $R_2 = \{(W,C_1,U,C_2)$: $\boldsymbol{W+W + C_1 = U + 10*C_2}\}$
  - $R_3 = \{(T,C_2,O,C_3)$: $\boldsymbol{T+T + C_2 = O + 10*C_3}\}$
  - $R_4 = \{(F,C_3)$: $\boldsymbol{C_3 = F}\}$
  - $R_5 = \{(T,W,O,R,U,F)$: $\boldsymbol{T \neq W \neq O \neq R \neq U \neq F}\}$

Restricción "global": *todos diferentes*

CSP no binario como hipergrafo

Los CSPs con restricciones k-arias (k>2) se representan como **hipergrafos**

- Cada variable x_i del CSP es representada por un nodo
- Cada restricción k-área R_i se representa por un **hiperarco** que conecta k nodos
- Ejemplo: problema de criptoaritmética



- Cualquier CSP con restricciones k-áreas (k>2) se puede representar mediante un CSP que contiene únicamente restricciones binarias
  - Para la restricción "todos diferentes" (R_5) esto es obvio, para los demás no tanto…

Problemas de Satisfacción de Restricciones

Solución a un CSP

- Una ***asignación*** es una función parcial $\sigma: X \rightarrow D(X)$
  - Habitualmente se escribe "$x_1 = v_a$" por "$\sigma(x_1) = v_a$" y se describe una asignación de forma extensiva, es decir $\sigma = \{x_1 = v_a , x_3 = v_b , x_7 = v_c , \ldots\}$
  - Una asignación es *completa* si está definida para todas las variables X del CSP
  - Una asignación σ *cumple* con una restricción R_i sii

$$(\sigma(x_1),\ldots, \sigma(x_l)) \in R_i(x_1,\ldots, x_l)$$

- Una asignación completa σ es una ***solución*** si cumple todas las restricciones en $R_i \in R$
- *Ejemplos* para el caso de las 4 reinas
  - $\sigma = \{x_1 = 2 , x_2 = 4 , x_3 = 3 \}$ es una asignación parcial que cumple R'_2
  - $\sigma = \{x_1 = 1 , x_2 = 2 , x_3 = 3 , x_4 = 4 \}$ es una asignación total que no cumple R'_2
  - $\sigma = \{x_1 = 2 , x_2 = 4 , x_3 = 1 , x_4 = 3 \}$ es una solución al problema de las 4 reinas

Módulo II: Agentes basados en Búsqueda

Tema 2: Búsqueda no-informada

Tema 3: Búsqueda heurística

Tema 4: Búsqueda heurística avanzada

Tema 5: Búsqueda multiagente

Tema 6: Búsqueda con espacios estructurados

- CSPs
- Satisfacción de restricciones
- Propagación de restricciones

Satisfacción de restricciones:
Métodos de búsqueda para encontrar una solución al CSP

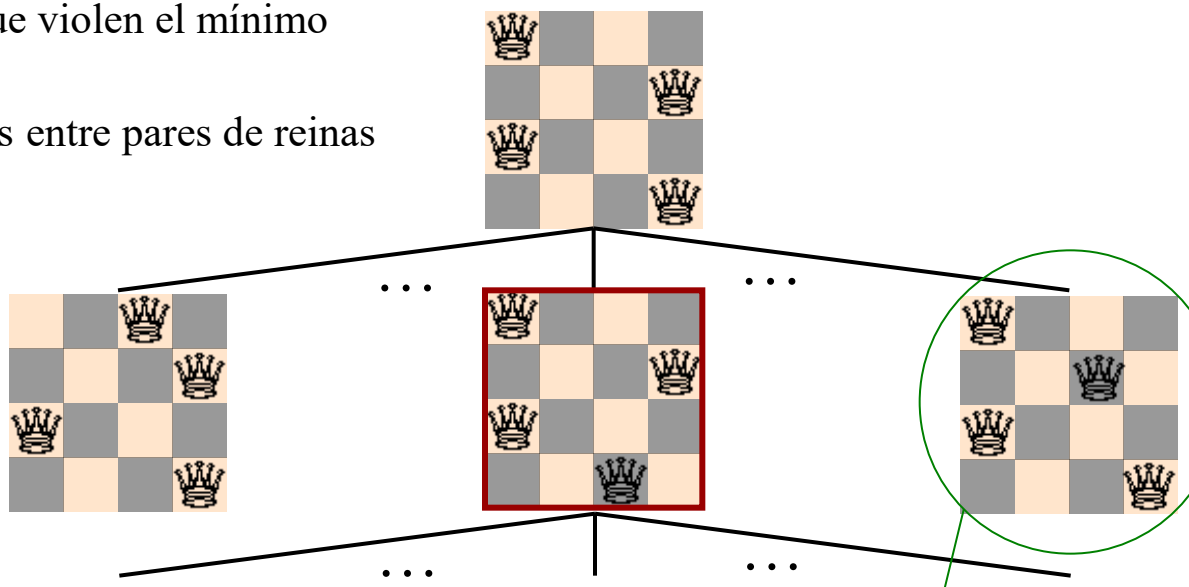
Búsqueda con asignaciones *completas*

- *Estado*: asignación completa (valores para *todas* las variables)
- *Operador*: modificar el valor de una variable en la asignación
- *Meta?*: asignación que *cumple* todas las restricciones
- *Coste*: cero (la longitud de un camino hasta un nodo meta es irrelevante)
- *Heurística del conflicto mínimo*:
  - Preferir los sucesores que violen el mínimo número de restricciones
  - 4-reinas: n° de amenazas entre pares de reinas

columna 3

fila 2

3	1	3	3
4	5	2	3
3	2	5	4
3	3	1	3



Significado de la tabla. Ejemplo:
"En el estado que resulta de colocar la reina de la fila 2 en la columna 3 hay 2 pares de reinas que se amenazan"

Solución con métodos de búsqueda

Nótese:

- En cada camino del árbol de búsqueda, tiene sentido cambiar el valor de cada variable como mucho 1 vez: el árbol de búsqueda tiene profundidad máxima n
- A cada una de las n variables se pueden asignar d valores: al nivel 0 hay $n \cdot d$ sucesores; al nivel 1 hay $(n\text{-}1) \cdot d$ sucesores; etc…
- Por tanto, se genera un árbol con hasta $n! \cdot d^n$ hojas – pero sólo hay d^n posibles asignaciones!
- Equivalencia de caminos por *conmutatividad:* el *orden* en el que se cambian los valores de las variables es irrelevante

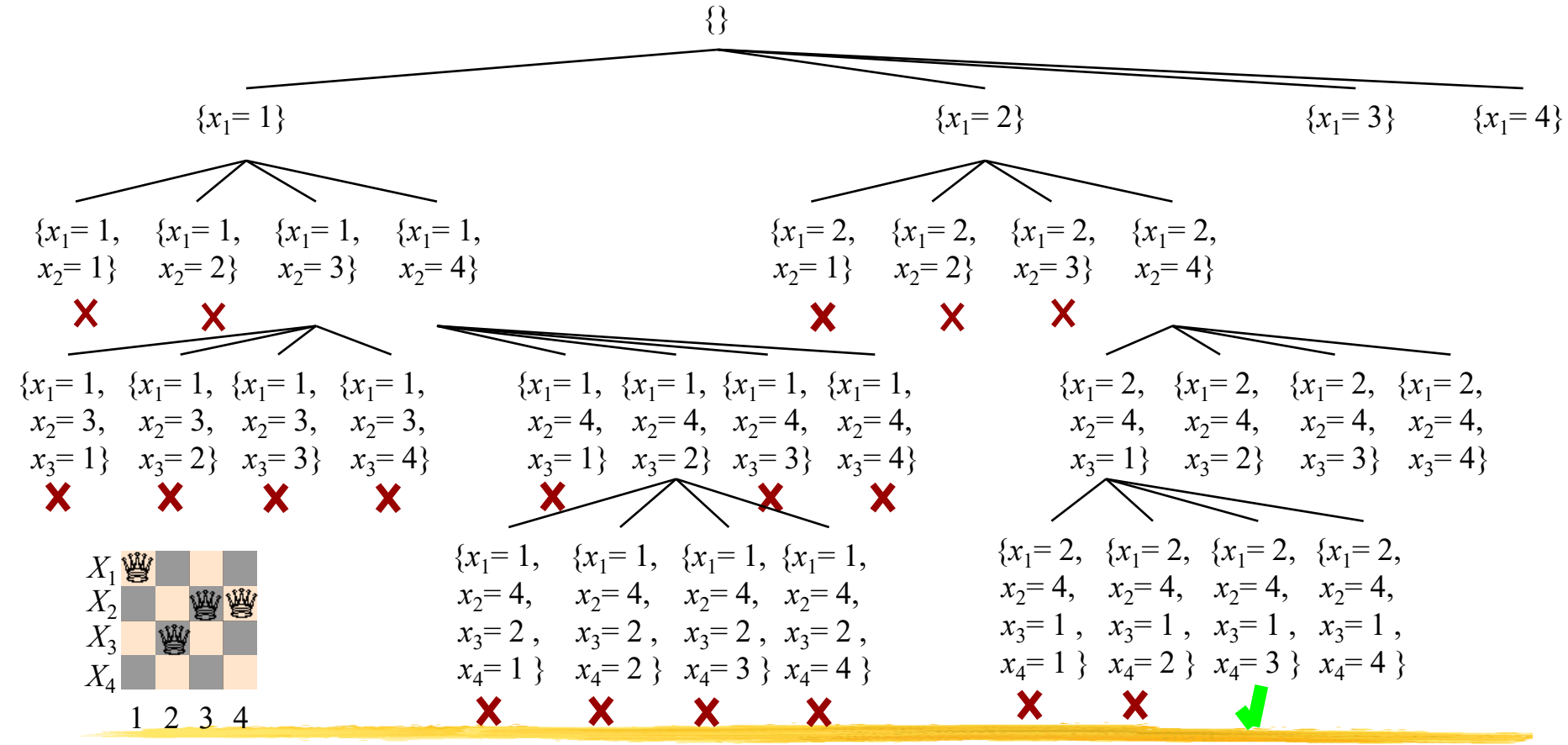
Búsqueda con asignaciones *parciales*

- *Estado*: asignación parcial (valores asignados a *algunas* variables)
- *Operador*: elegir un valor para una variable *no asignada*
  (el posible estado sucesor ha de cumplir R)
- *Meta?*: solución (asignación *completa* que cumple *todas* las restricciones)
- *Coste*: cero (la longitud de un camino hasta un nodo meta es irrelevante)

Satisfacción de restricciones con vuelta atrás cronológica

Algoritmo: **Vuelta atrás cronológica (*cronological backtracking*)**

- Búsqueda en profundidad en el espacio de asignaciones parciales
- Ejemplo: 4-reinas



Satisfacción de restricciones con vuelta atrás cronológica

Función vuelta-atrás-cronológica(CSP, σ) **devuelve** σ' / *fallo*
  Si |σ| = *n* **entonces** % la asignación es completa (i.e. una solución)
    devolver(σ)
  $x_i \leftarrow$ ***elegir-variable-no-asignada***(X, σ) % orden de elección no afecta la completitud
  dominio $\leftarrow D_{Xi}$
  Mientras dominio ≠ {} **hacer** % en el dominio quedan valores por probar
    $v \leftarrow$ **elegir-valor**(*dominio*)
    $\sigma' \leftarrow \sigma \cup \{x_i = v\}$ % generar nueva asignación parcial
    Si σ' ∈ *R* **entonces** % la nueva asignación parcial es consistente
      resultado ← vuelta-atrás-cronológica(CSP, σ') % completarla
      Si resultado ≠ *fallo* **entonces**
        devolver(resultado)
    dominio ← dominio \ {*v*}
  Fin {Mientras}
  devolver(*fallo*)
Fin {vuelta-atrás-cronológica}

Heurísticas

Nótese

- La selección de variables (*elegir-variable-no-asignada*) y de valores (*elegir-valor*) no afecta a la completitud (el espacio de búsqueda es finito)
- pero *sí* afecta a la complejidad del algoritmo (el tamaño del árbol de búsqueda)

Heurísticas para *elección de variables*

- Hay que asignar *todas* las variables del CSP
- Preferir variables "difíciles": detectar posibles 'callejones sin salida' (*fail first*)
  - Preferir la variable de **mínimos valores restantes** *(mimimum remaining values, MRV)*
  - Preferir la variable con mayor número de restricciones (**grado heurístico**)

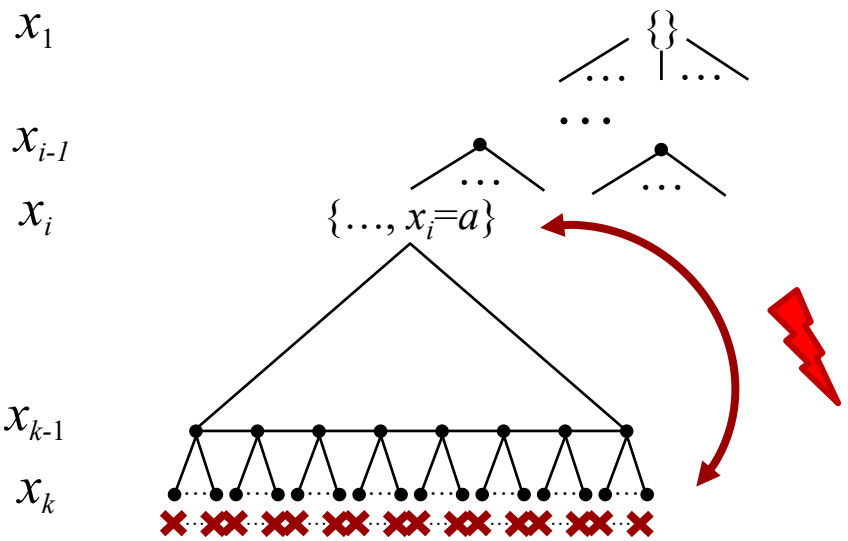
Heurísticas para *elección de valores*

- Es suficiente con asignar *uno* de los valores en el dominio de una variable
- Preferir valores "prometedores": generar una solución cuanto antes (*fail last*)
  - Preferir el **valor menos limitante** (respecto a variables no asignadas)
  - ...

Problema: "Thrashing"

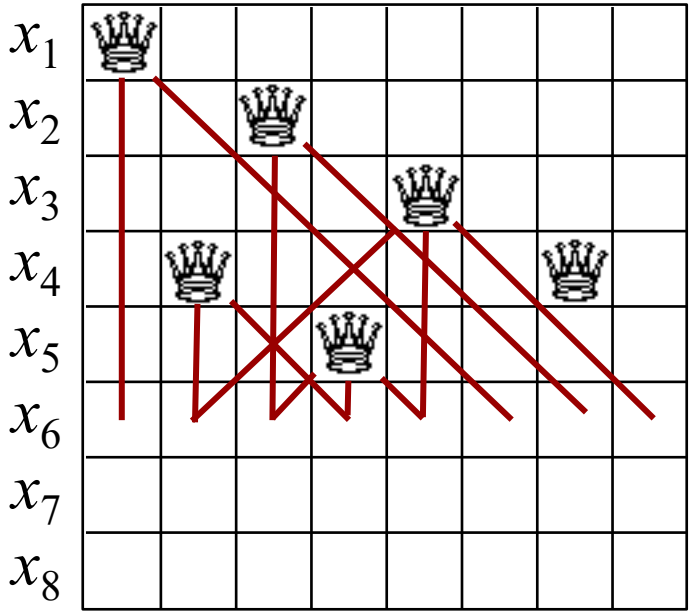
Problema del algoritmo de vuelta atrás cronológica (*cronological backtracking*):

- No se toma en cuenta la "**causa**" de haber llegado a un callejón sin salida
- **"Thrashing"**: se genera el mismo error de asignación (subárbol) varias veces
- Ejemplo:
  - $x_i = a$ es incompatible con cualquier valor del dominio de x_k
  - sin embargo, se irían probando los posibles valores para x_k (en combinación con $x_i = a$) un número exponencial de veces



Mejoras al *cronological backtracking*

- Satisfacción de restricciones con salto atrás (*backjumping*):
  - en lugar de retroceder a la variable más recientemente instanciada que todavía tiene alternativas disponibles
  - continuar la búsqueda en la primera variable que causó el fallo
  - Ejemplo: 8-reinas
    - No hay valor posible para x_6
    - Revisar el valor de x_5 no resuelve el problema
    - Retroceder directamente a x_4 (saltándose x_5)



- *Nogoods*:
  - Recordar las asignaciones parciales que han causado fallo para que no se vuelvan a generar
  - Ej. transparencia anterior: evitar que se vuelva a generar una asignación con $x_i = a$

Módulo II: Agentes basados en Búsqueda

Tema 2: Búsqueda no-informada

Tema 3: Búsqueda heurística

Tema 4: Búsqueda heurística avanzada

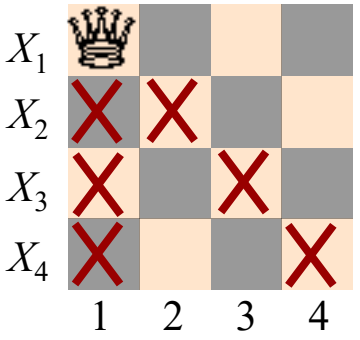
Tema 5: Búsqueda multiagente

Tema 6: Búsqueda con espacios estructurados

- CSPs
- Satisfacción de restricciones
- Propagación de restricciones

Propagación de restricciones

- Idea: usar el conjunto de restricciones para reducir el espacio de búsqueda
- Método:
  - "propagar" valores a través de las restricciones que afectan a sus variables,
  - intentando eliminar de los dominios los valores que no podrán formar parte de una solución
- Ejemplo de las 4 reinas: Si $\{x_1 = 1\}$ se puede
  - eliminar 1 y 2 de D_2 , i.e $D_2 = \{3,4\}$ (debido a $R'_1(x_1,x_2)$)
  - eliminar 1 y 3 de D_3 , i.e $D_3 = \{2,4\}$ (debido a $R'_2(x_1,x_3)$)
  - eliminar 1 y 4 de D_4 , i.e $D_4 = \{2,3\}$ (debido a $R'_3(x_1,x_4)$)



Aplicación:

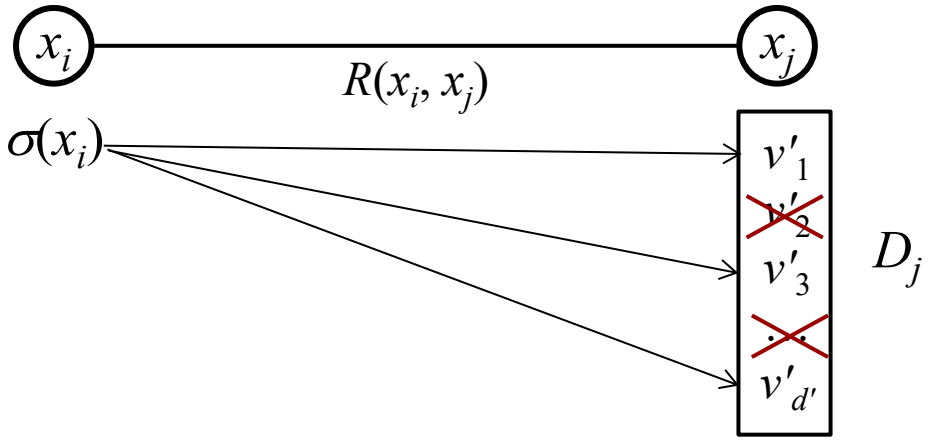
a) *Antes* de la búsqueda, para reducir la cardinalidad de los dominios

b) *Intercalado* con la búsqueda: filtrar dominios sobre la base de una asignación parcial

Propagación de restricciones: comprobación hacia adelante

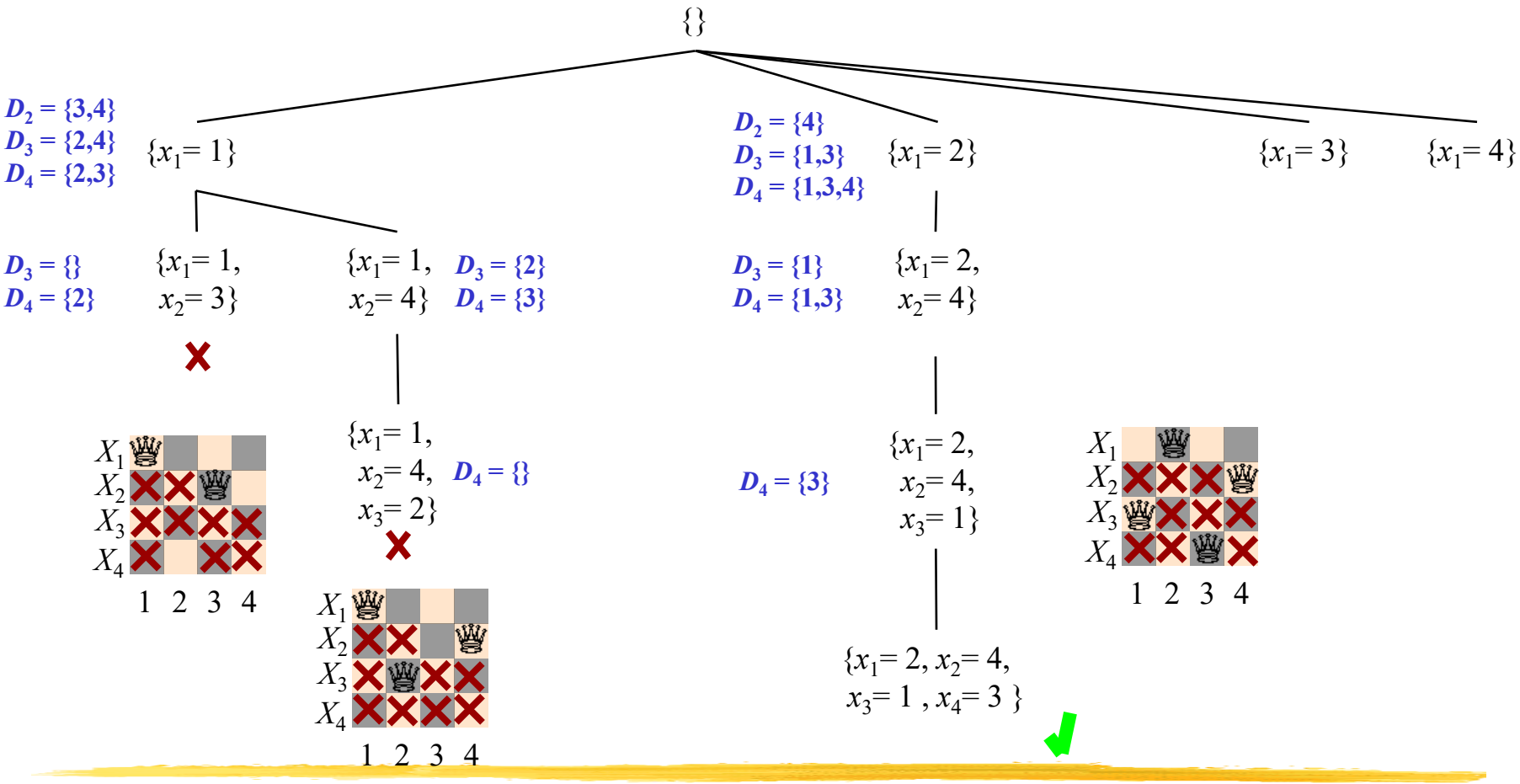
Búsqueda con vuelta atrás cronológica con comprobación hacia delante (*forward checking*):

- Filtrar dominios después de cada ampliación de la asignación parcial σ:
  - Para todas las restricciones binarias R que involucran una variable x_i asignada en σ y otra x_j no asignada en σ
  - Para todo valor v' de D_j : Si $(\sigma(x_i), v')$ viola $R(x_i, x_j)$ entonces eliminar v' de D_j
- Marcha atrás si algún dominio se queda sin valores

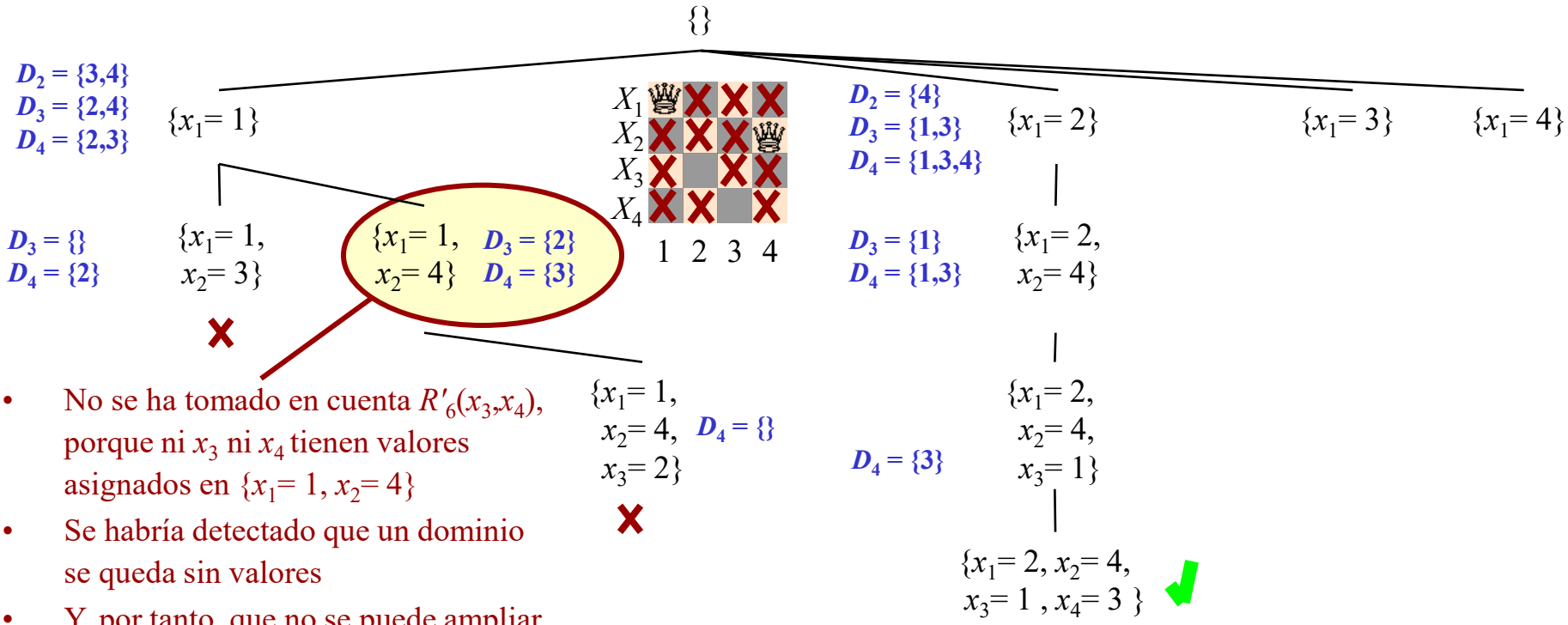


Propagación de restricciones: comprobación hacia adelante

Ejemplo: 4 reinas



Transitividad de Restricciones



- No se ha tomado en cuenta $R'_6(x_3,x_4)$, porque ni x_3 ni x_4 tienen valores asignados en $\{x_1 = 1, x_2 = 4\}$
- Se habría detectado que un dominio se queda sin valores
- Y, por tanto, que no se puede ampliar la asignación para que forme una solución

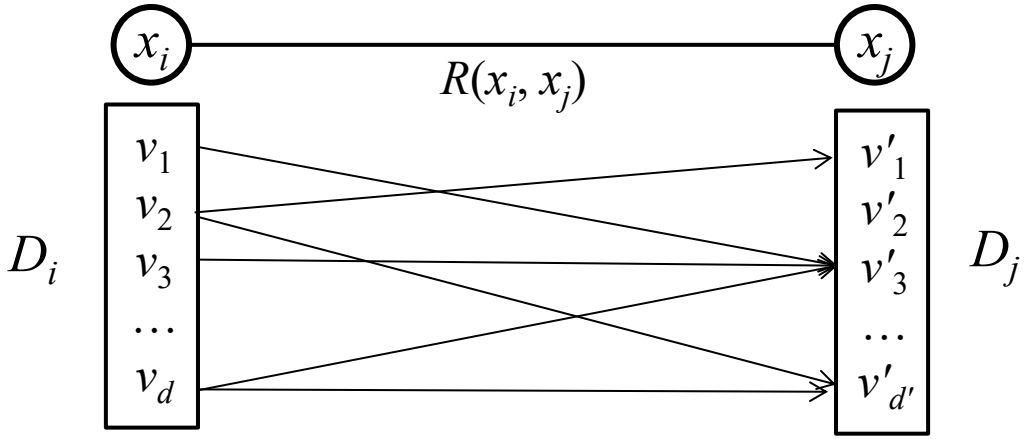
- Hay que ir más allá del *forward checking* y tomar en cuenta la *transitividad* de las restricciones…

Consistencia de arco

Definición:

- Una variable x_i es **arco consistente** con respecto a una restricción binaria $R(x_i, x_j)$, si

$$\forall v \in D_i \; \exists v' \in D_j \;\; (v, v') \, cumple \; R(x_i, x_j)$$



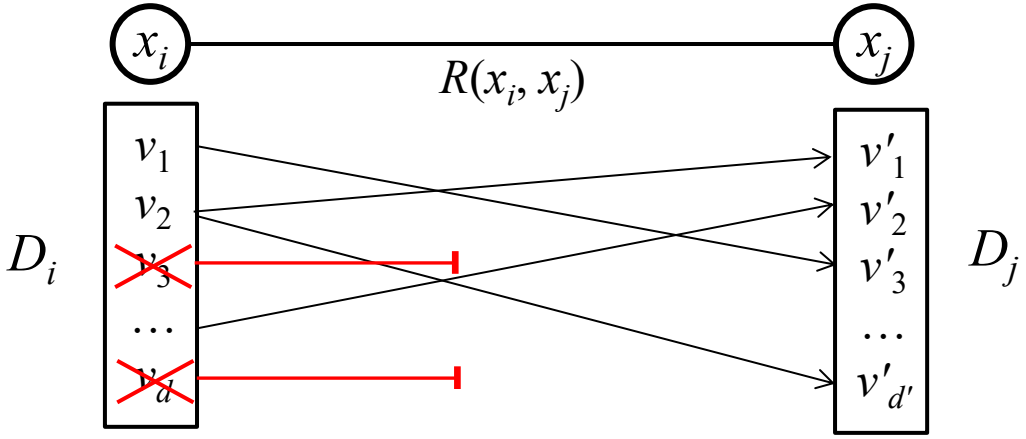
Extensión del concepto:

- Una variable es **arco-consistente**, si lo es con respecto a todas las restricciones binarias $R(x_i, _)$ en las que está involucrada
- Una restricción binaria $R(x_i, x_j)$ es **arco-consistente**, si tanto x_i como x_j son arco consistentes con respecto a ella
- Un CSP es **arco-consistente**, si todas sus restricciones son arco-consistentes

Consistencia de arco

Nótese:

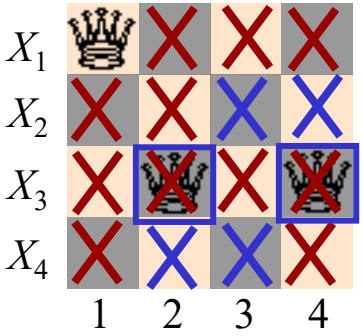
- Si una variable x_i NO es **arco consistente** con respecto a una restricción binaria $R(x_i, x_j)$, entonces existe al menos un valor $v \in D_i$ que es incompatible con todos los valores $v' \in D_j$ con respecto a $R(x_i, x_j)$
- Eliminando estos valores v NO modifica el conjunto de soluciones de un CSP



Consistencia de arco

Ejemplo:

- CSP de cuatro reinas donde la reina de la fila 1 se ha colocado en la columna 1, e.d. con $D_1 = \{1\}$, $D_2 = \{3,4\}$, $D_3 = \{2,4\}$, y $D_4 = \{2,3\}$
- El CSP **no** es arco consistente, entre otras, por la variable x_3
  - Para $x_3 = 2$ no existe $v' \in D_4$ tal que $(2,v') \in R'_6(x_3,x_4))$
  - Para $x_3 = 4$ no existe $v' \in D_2$ tal que $(4,v') \in R'_4(x_3,x_2))$
- Por tanto, se pueden **eliminar** los valores 2 y 4 de D_3
- Al hacer x_3 arco-consistente, D_3 se queda vacío, y se concluye que el problema de las 4 reinas con $x_1 = 1$ **no tiene solución**

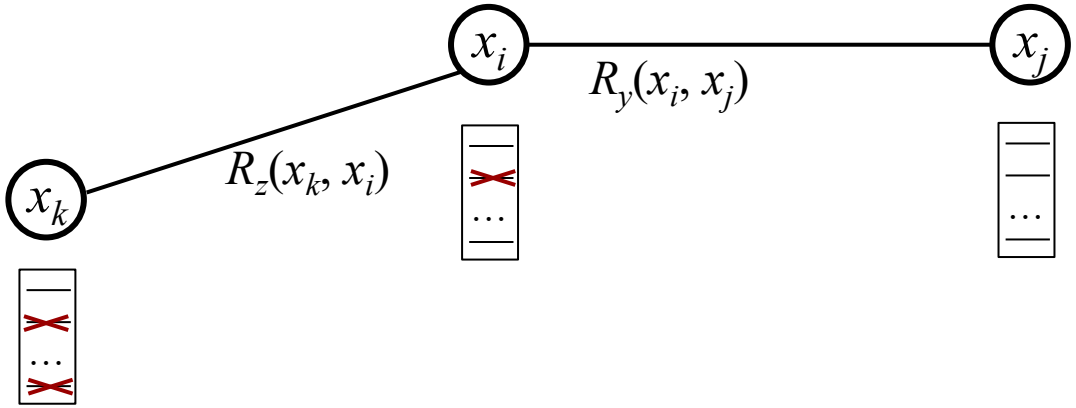


Def. arco.-consistencia: $\forall v \in D_i \; \exists v' \in D_j \;\; (v,v') \, cumple \; R(x_i, x_j)$

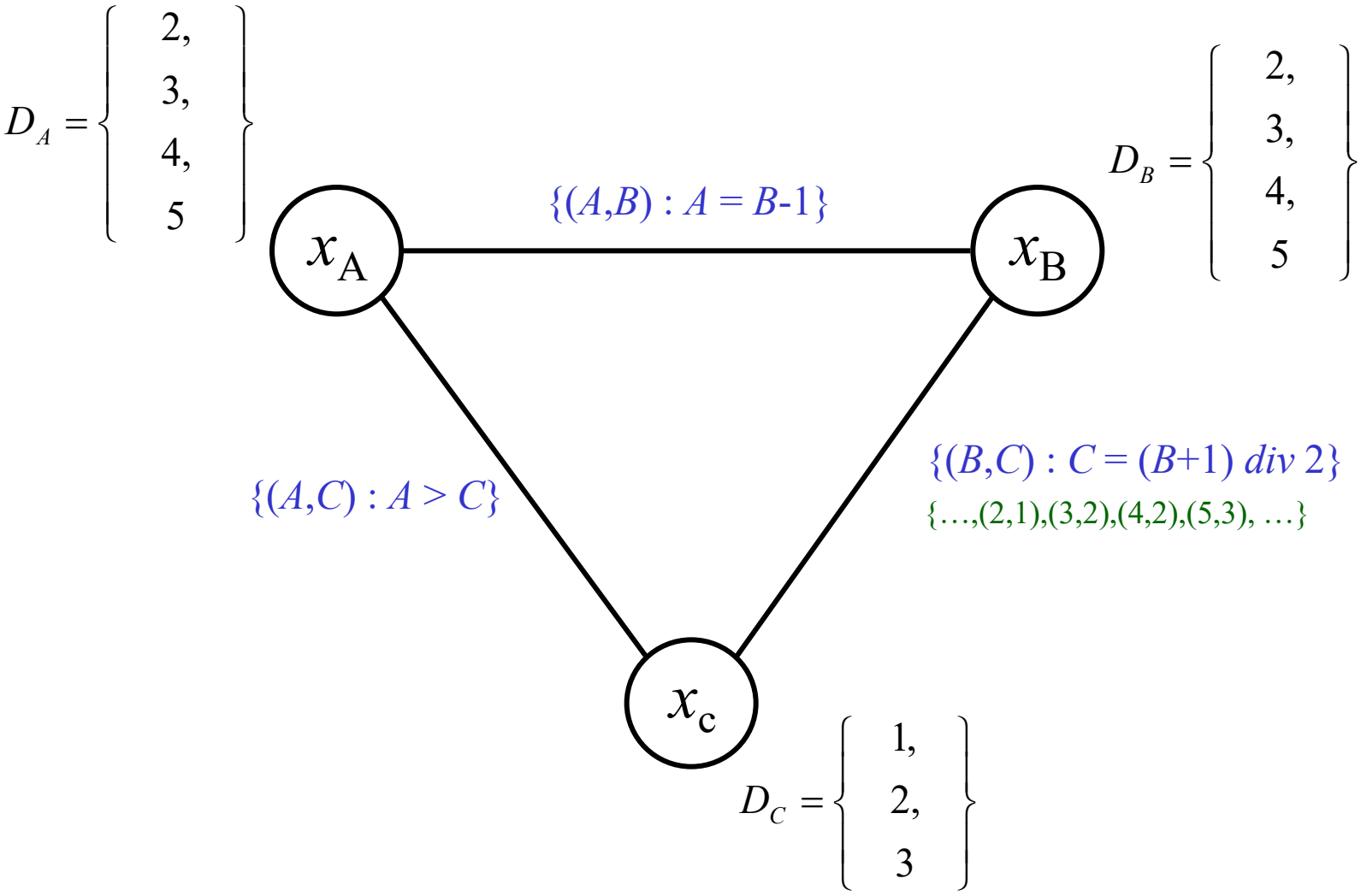
Consistencia de arco

Método para alcanzar arco-consistencia en un CSP:

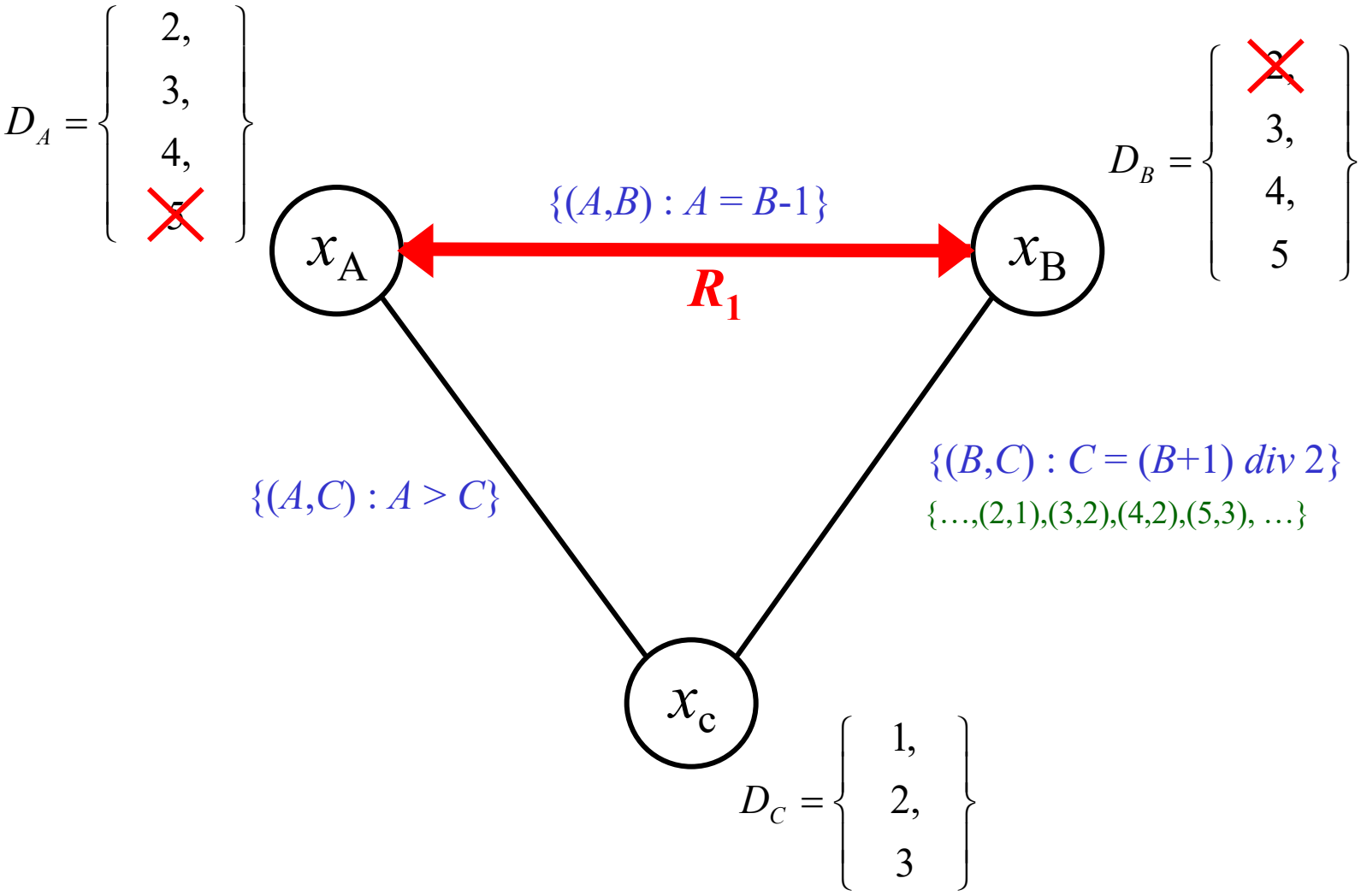
- Para todas las variables x y restricciones $R(x,_)$ en las que x está involucrada
  - Hacer x arco-consistente con respecto a $R(x,_)$
- Al eliminar un valor del dominio D_i de una variable x_i involucrada en la restricción $R_y(x_i,x_j)$, se puede "estropear" la arco consistencia de otra variable x_k con respecto a la restricción $R_z(x_i,x_k)$,
- Es decir, aunque ya se haya alcanzado arco-consistencia para x_k con respecto a la restricción $R_z(x_i,x_k)$, es posible que haya que volver a hacerlo (e.d. puede ser necesario "propagar" valores varias veces por la misma restricción)



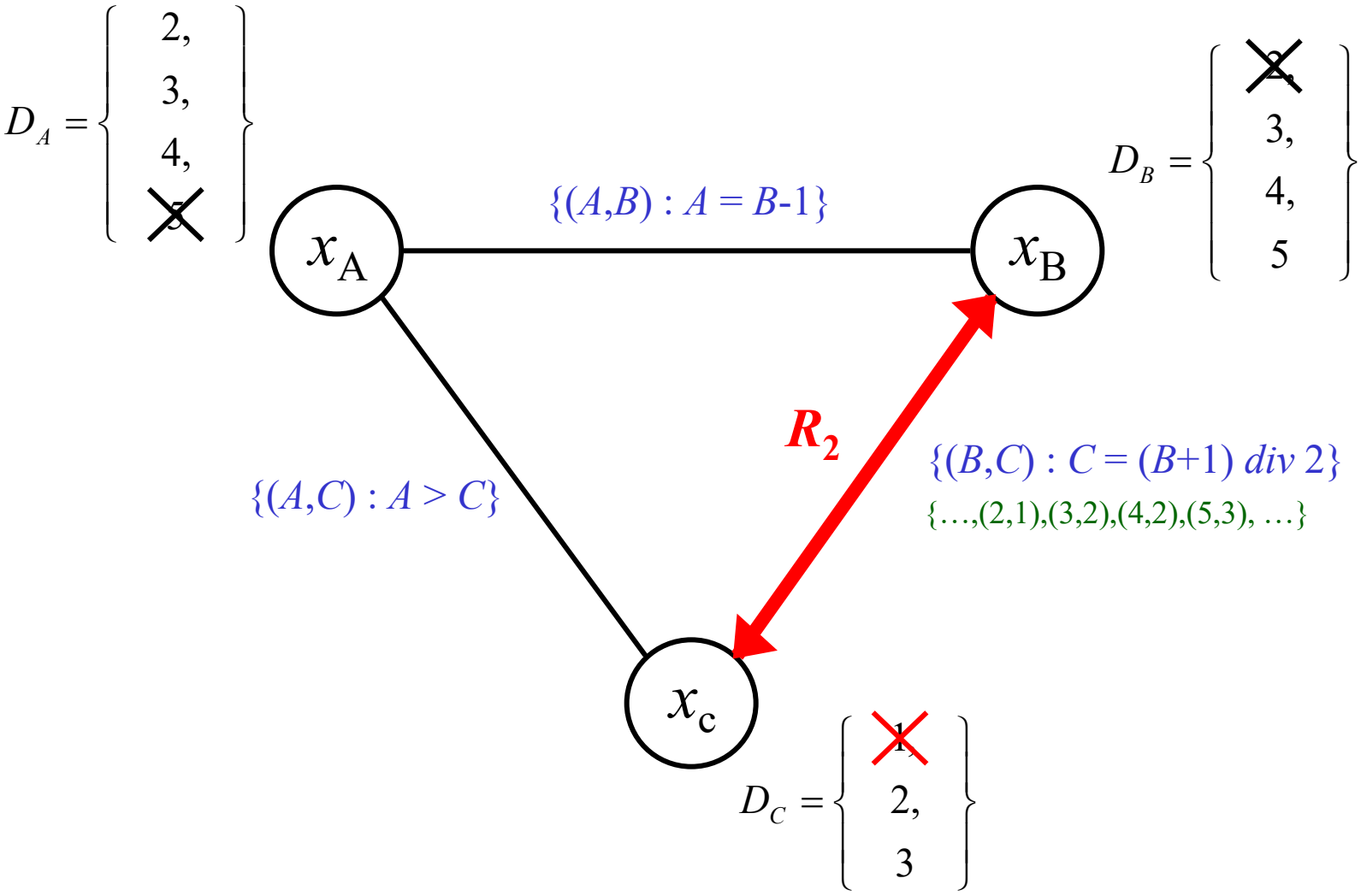
Algoritmo de arco consistencia



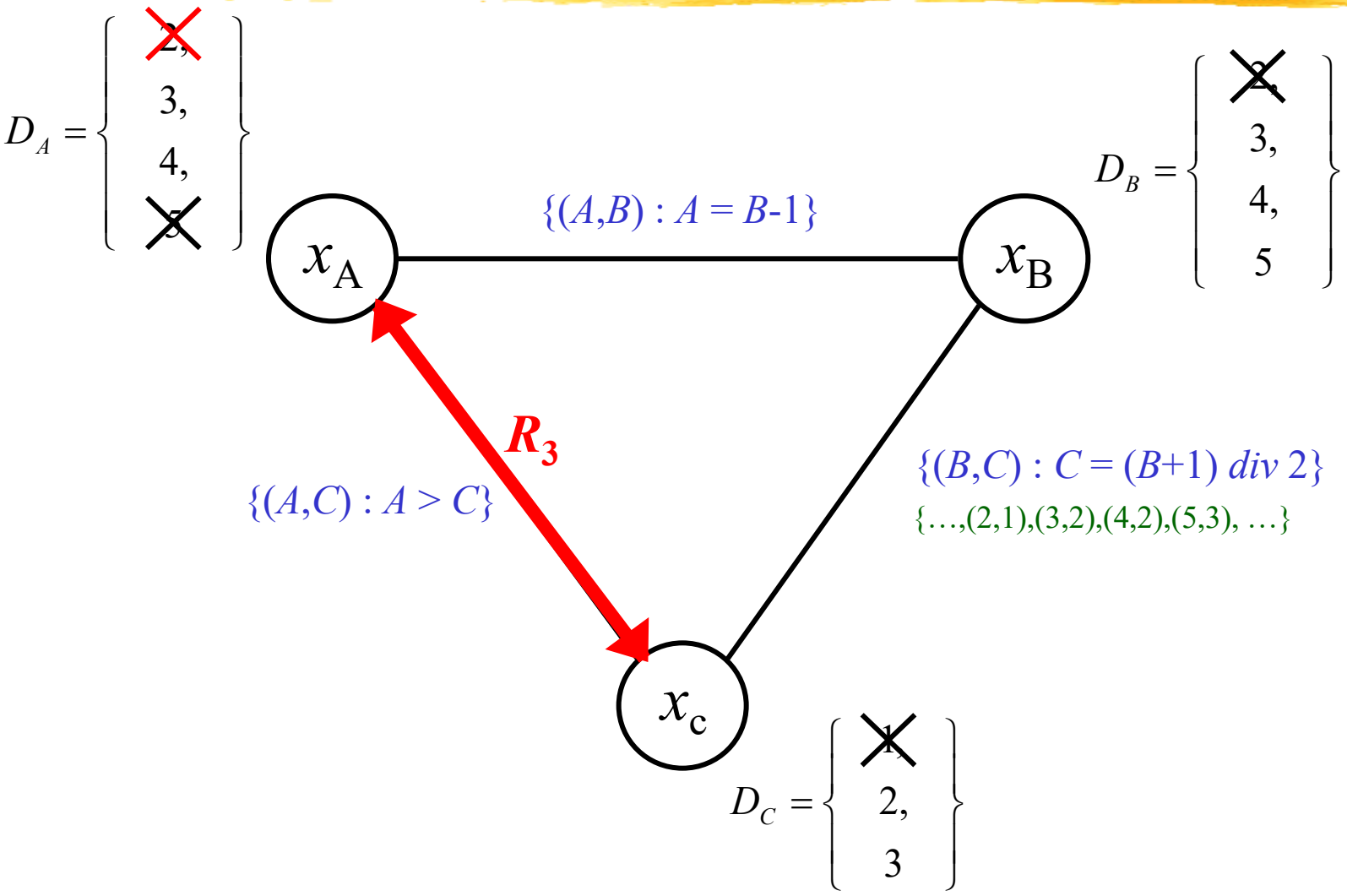
Algoritmo de arco consistencia



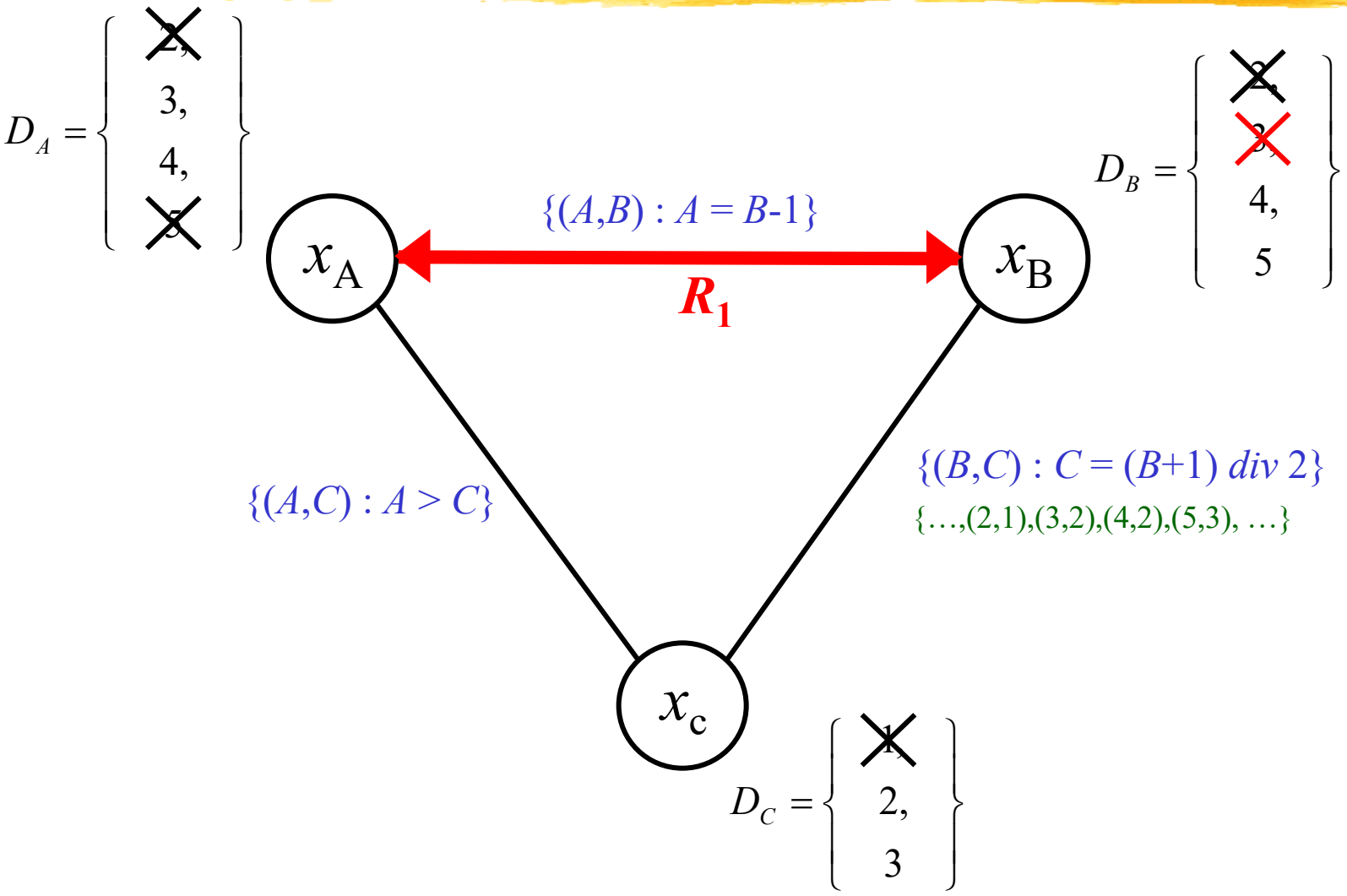
Algoritmo de arco consistencia



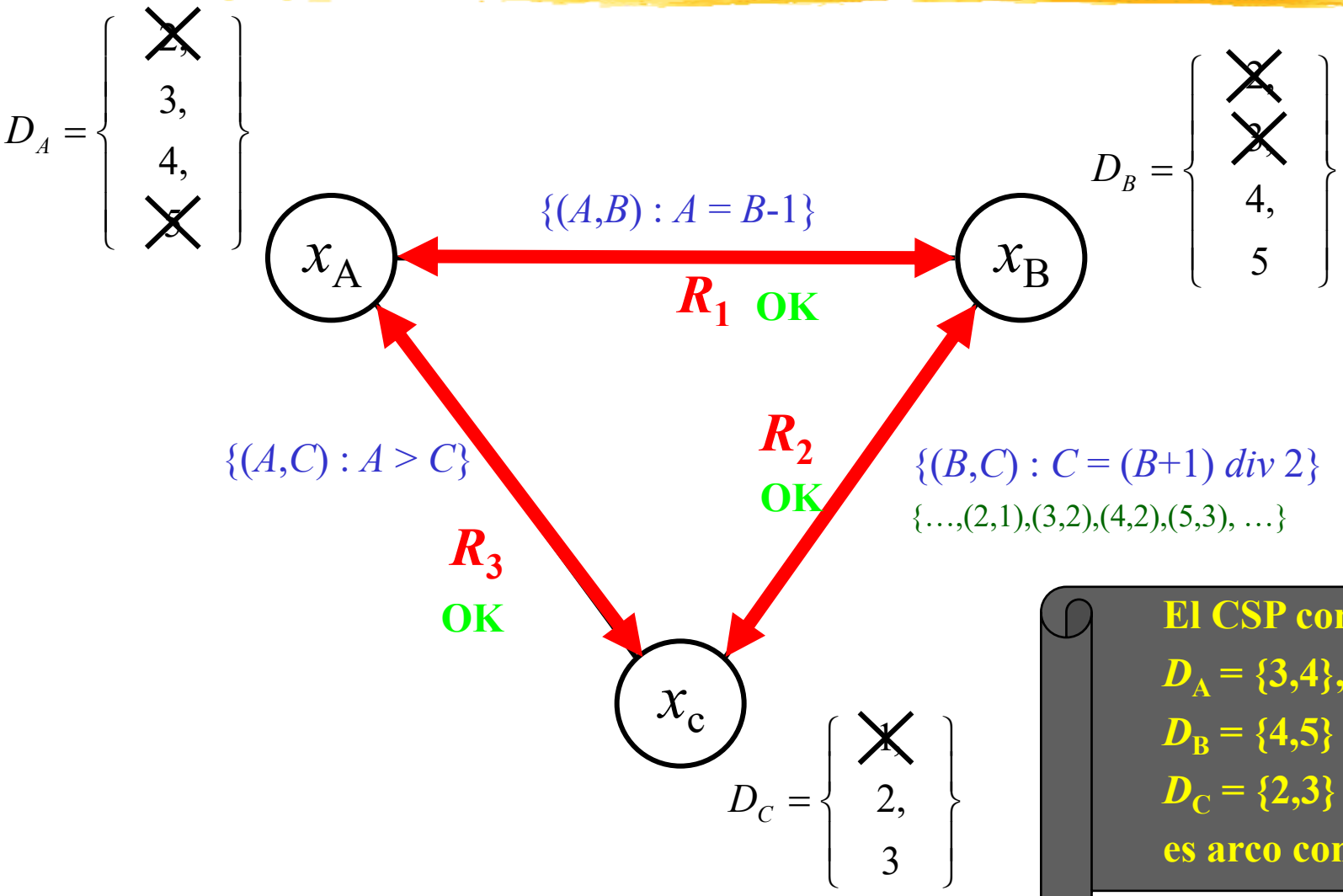
Algoritmo de arco consistencia



Algoritmo de arco consistencia



Algoritmo de arco consistencia



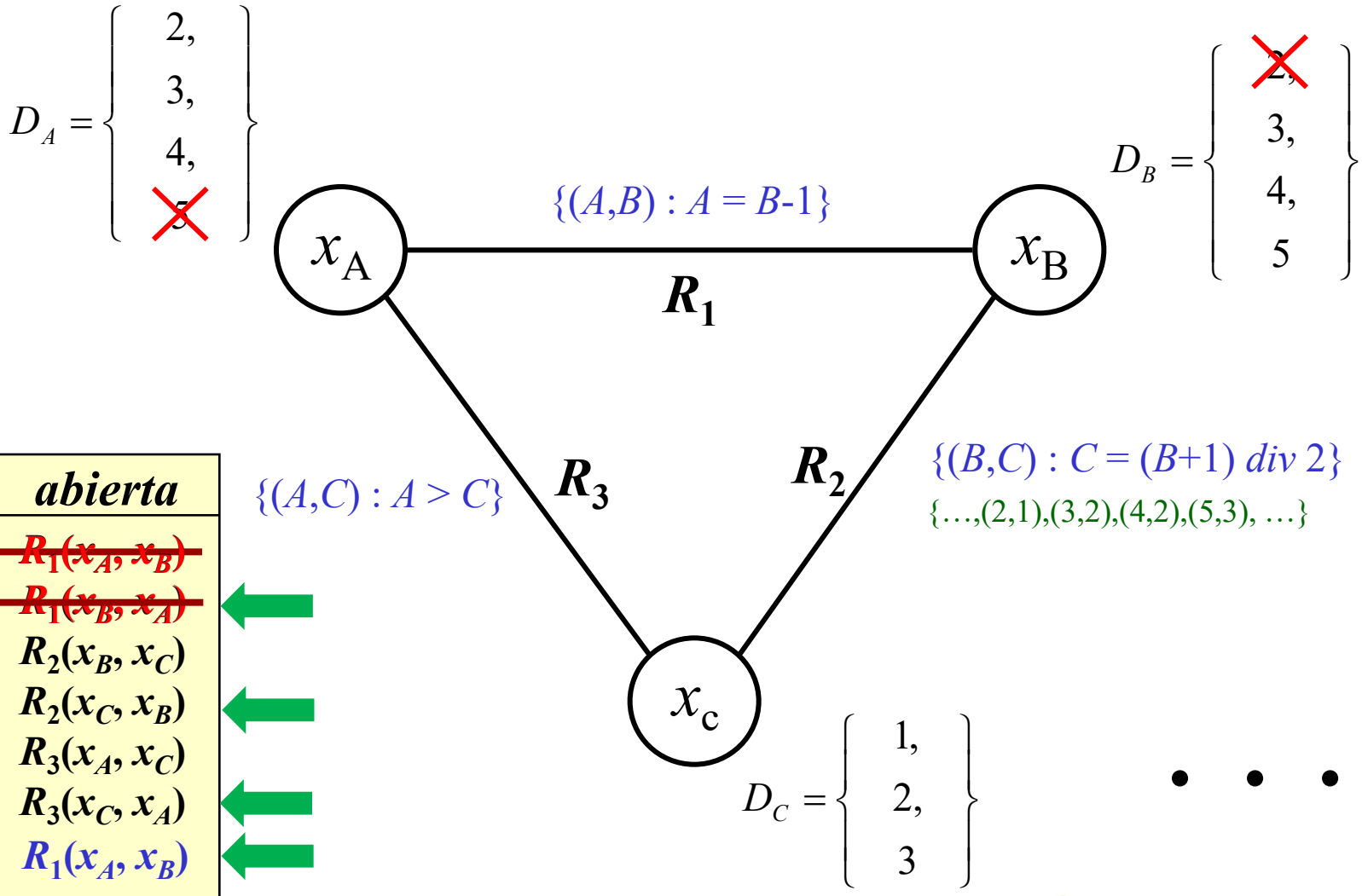
El CSP con
D_A = {3,4},
D_B = {4,5} y
D_C = {2,3}
es arco consistente

Algoritmo de arco consistencia

Función consistencia-de-arcos(CSP) **devuelve** CSP' % con dominios reducidos
  abierta ← R % inicialmente todas las restricciones binarias *dirigidas*
  Mientras ¬*vacio?*(abierta) **hacer**
    $R_y(x_i, x_j)$ ← *primero*(abierta)
    Si borrar-valores-inconsistentes($R_y(x_i, x_j)$) **entonces**
      Para cada $x_k \in$ vecinos(x_i) **hacer**
        abierta ← abierta ∪ {$R_z(x_k, x_i)$ }
Fin {consistencia-de-arcos}

Función borrar-valores-inconsistentes($R_y(x_i, x_j)$) **devuelve** *si* / *no*
  borrado ← *no* % variable booleana – iniciar con *false*
  Para cada $v \in D_i$ **hacer** % borrar valores inconsistentes (sólo de D_i)
    Si ¬∃ $v' \in D_j$ tal que $(v,v') \in R_y(x_i, x_j)$ **entonces**
      $D_i \leftarrow D_i \setminus \{v\}$
      borrado ← *si*
  Devolver (borrado)
Fin {borrar-valores-inconsistentes}

Algoritmo de arco consistencia



abierta
~~$R_1(x_A, x_B)$~~
~~$R_1(x_B, x_A)$~~
$R_2(x_B, x_C)$
$R_2(x_C, x_B)$
$R_3(x_A, x_C)$
$R_3(x_C, x_A)$
$R_1(x_A, x_B)$

Algoritmo de arco consistencia

Resultado:

- El algoritmo de consistencia de arco reduce un CSP en un CSP' *equivalente* (es decir: con las mismas soluciones)
- Si al aplicar el algoritmo el dominio de una variable se queda vacío, el CSP es *inconsistente* (es decir: no tiene solución)

Complejidad:

- Un grafo que representa el CSP binario tiene como mucho $n \cdot (n-1)$ (i.e., $O(n^2)$) arcos (dirigidos)
- Cada arco puede insertarse en *abierta* a lo sumo d veces
- El borrado de valores que no son arco-consistentes se realiza en $O(d^2)$ pasos
- Por tanto, la complejidad en tiempo en el peor caso es $O(n^2 \cdot d^3)$

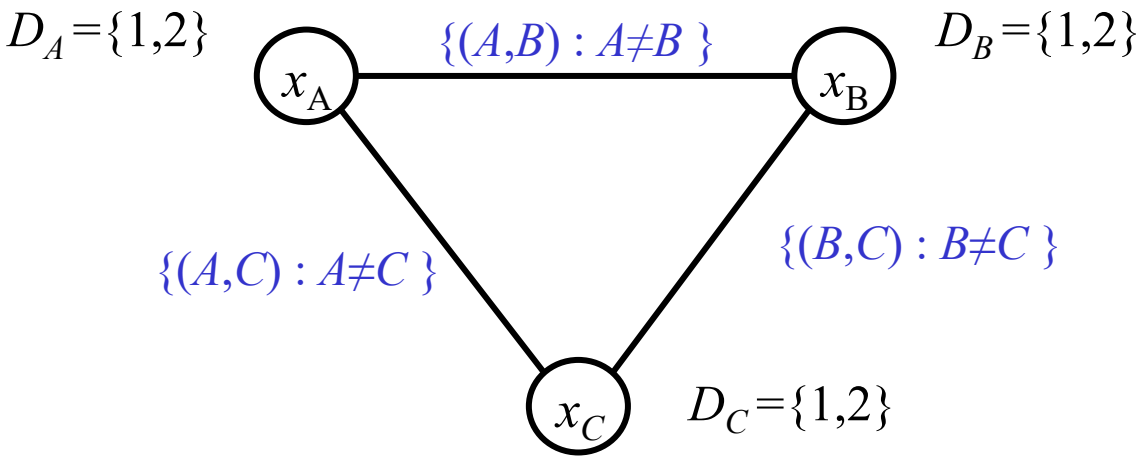
Pero:

- La reducción en el tamaño de los dominios (y el ahorro correspondiente en el tiempo de ejecución del algoritmo de satisfacción de restricciones) suele compensar el coste adicional de aplicar el algoritmo de propagación de restricciones para alcanzar arco-consistencia

Algoritmo de arco consistencia

Análisis:

- No se puede garantizar que el algoritmo de arco consistencia detecte *cualquier* CSP inconsistente



- ***k*-consistencia**: para cualquier asignación consistente σ_{k-1} a k-1 variables, y para cualquier variable restante, existe un valor en el dominio de esta k-ésima variable que es consistente con σ_{k-1}
- Arco-consistencia se conoce también como **2-consistencia**
- El ejemplo de arriba NO es **3-consistente** (también llamado "camino-consistente")
- Se puede asegurar que un CSP con n variables tiene solución si es **fuertemente *n*-consistente** (k-consistente para $k = 1..n$)

Algoritmo MAC

Algoritmo: Mantenimiento de consistencia de arcos

- Inglés: *Maintaining Arc Consistency* (MAC)
  - aka: *Full Look-Ahead* (FLA)
- Intercalar satisfacción y propagación de restricciones
  - Realizar búsqueda con vuelta atrás cronológica
  - Después de aumentar una asignación, construir un CSP equivalente que sea arco-consistente
- MAC es de los algoritmos básicos más conocidos para CSPs

Algoritmo MAC

Función MAC(CSP, σ) **devuelve** σ' / *fallo*

Si |σ| = *n* **entonces** % la asignación es completa (i.e. una solución)

devolver(σ)

$x_i \leftarrow$ elegir-variable-no-asignada(X, σ) % orden de elección no afecta la completitud

dominio $\leftarrow D_{Xi}$

Mientras dominio ≠ {} **hacer** % en el dominio quedan valores por probar

$v \leftarrow$ elegir-valor(*dominio*)

σ' $\leftarrow$ σ $\cup$ {$x_i = v$} % generar nueva asignación parcial

Si σ' $\in$ *R* **entonces** % la nueva asignación parcial es consistente

CSP' $\leftarrow$ consistencia-de-arcos(σ'(CSP))

resultado $\leftarrow$ **MAC**(**CSP'**, σ') % completarla

Si resultado ≠ *fallo* **entonces**

devolver(resultado)

dominio $\leftarrow$ dominio \ {v}

Fin {Mientras}

devolver(*fallo*)

Fin {MAC}

CSPs avanzados

Temas avanzados:

- Algoritmos de satisfacción y propagación más sofisticados
- Heurísticas de selección de variables y/o valores más sofisticadas
- CSPs con dominios *infinitos* y *continuos*
- Estructuración (jerárquica) de CSPs
- Preferencias sobre soluciones (*optimización* de restricciones, COPs)
- Satisfacción /optimización *distribuida* de restricciones (DCSP/DCOP)
- *Relajación* de restricciones
- . . .